

Exp 8 — Model Evaluation and Improvement

Aim

To demonstrate **hyperparameter tuning using Grid Search** and **5-fold Cross-Validation** for improving and evaluating an SVM model.

Algorithm

1. Load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Standardize the features.
4. Define the SVM model and hyperparameter grid.
5. Perform Grid Search with 5-fold Cross-Validation.
6. Select the best hyperparameters.
7. Evaluate the tuned model using accuracy, classification report and confusion matrix.
8. Perform 5-fold Cross-Validation.
9. Calculate mean accuracy and standard deviation.
10. Visualize the cross-validation scores.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, KFold, cross_val_score, GridSearchCV

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from sklearn.preprocessing import StandardScaler
```

```
iris = load_iris()

X = iris.data

y = iris.target

target_names = iris.target_names

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": [1, 0.1, 0.01, 0.001],
    "kernel": ["rbf", "linear"]
}

grid_search = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train_scaled, y_train)
```

```
print("Best Hyperparameters:", grid_search.best_params_)

print("Best Cross-Validation Accuracy:", round(grid_search.best_score_, 4))

best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test_scaled)

print("Test Accuracy:", round(accuracy_score(y_test, y_pred), 4))

print("\nClassification Report:")

print(classification_report(y_test, y_pred, target_names=target_names))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))

sns.heatmap(

    cm,

    annot=True,

    fmt="d",

    cmap="Blues",

    xticklabels=target_names,

    yticklabels=target_names

)

plt.title("Confusion Matrix - Tuned SVM")

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.show()

model_cv = SVC(random_state=42)
```

```
k_folds = 5

kf = KFold(n_splits=k_folds, shuffle=True, random_state=42)

cv_scores = cross_val_score(

    model_cv,

    X_train_scaled,

    y_train,

    cv=kf,

    scoring="accuracy"

)

print("Cross-Validation Scores:", cv_scores)

print("Mean CV Accuracy:", round(np.mean(cv_scores), 4))

print("CV Standard Deviation:", round(np.std(cv_scores), 4))

plt.figure(figsize=(8, 5))

plt.bar(range(1, k_folds + 1), cv_scores)

plt.axhline(

    y=np.mean(cv_scores),

    linestyle="--",

    label=f"Mean Accuracy ( {np.mean(cv_scores):.4f} )"

)

plt.title("5-Fold Cross-Validation Accuracy Scores")

plt.xlabel("Fold Number")

plt.ylabel("Accuracy")

plt.ylim(0.8, 1.0)
```

```
plt.legend()

plt.grid(axis="y")

plt.show()
```

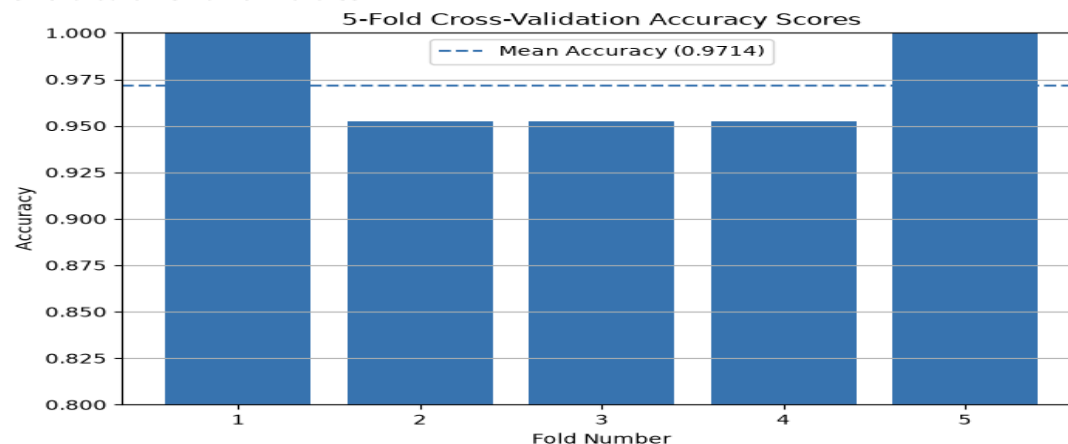
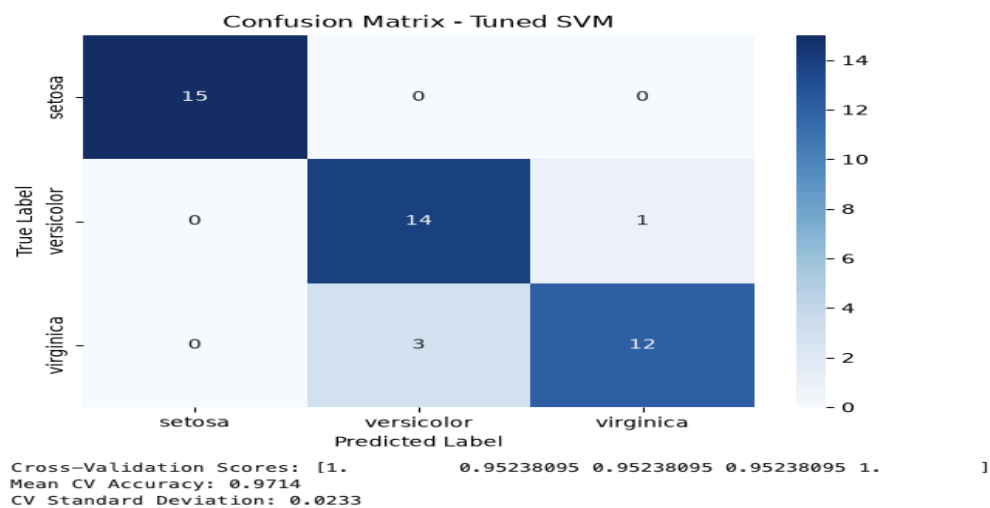
Output:

```
Best Hyperparameters: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best Cross-Validation Accuracy: 0.981
Test Accuracy: 0.9111
```

```
Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        15
  versicolor    0.82         0.93        0.88        15
   virginica     0.92         0.80        0.86        15

 accuracy         0.91
 macro avg        0.92         0.91         0.91         45
 weighted avg     0.92         0.91         0.91         45
```



Result:

The SVM model was successfully evaluated using **Grid Search and Cross-Validation**, and its performance was measured using accuracy and a confusion matrix.